

FIFO with Parity Checker

Fredrick Angelo Galapon

1 Introduction

This paper presents the design and implementation of a small logic block composed of a synchronous first-in, first-out (FIFO) memory and a parity checker.

2 System Overview

Figure 1 shows a block diagram of the system. The design consists of a first-in, first-out (FIFO) memory and a parity checker. The incoming data is first stored in the FIFO memory. The data read from the FIFO is then checked by the parity checker before being forwarded to the output channel. The following subsections describe each sub-block in detail.

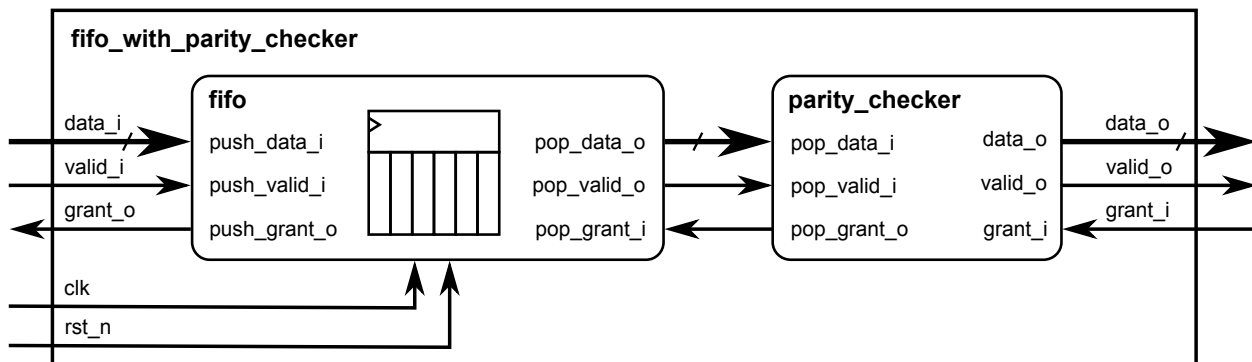


Figure 1: Block diagram of system.

2.1 FIFO Memory

Table 1 lists the external interface of the FIFO. The module operates synchronously with the global clock signal, `clk`. An active-low reset signal, `rst_n`, initializes the FIFO to an empty state. Data flow on both the push and pop interfaces is controlled through a valid-grant handshake protocol, where a transaction is completed only when both the valid and grant signals are asserted high. The FIFO module supports a configurable data width (`FifoWidth`) and depth (`FifoDepth`), and is capable of processing one transaction per clock cycle on both the push and pop interfaces simultaneously.

Table 1: Ports of the FIFO.

Port name	Direction	Width	Description
clk	in	1	global clock
rst_n	in	1	active-low asynchronous reset
push_data_i	in	FifoWidth	input data to be written into the FIFO
push_valid_i	in	1	indicates that the producer (sender) has valid data to send
push_grant_o	out	1	indicates that the FIFO is not full and can accept data
pop_data_o	out	FifoWidth	output data read from the FIFO
pop_valid_o	out	1	indicates that the FIFO has data available to be read
pop_grant_i	in	1	indicates that the consumer (receiver) is ready to receive data

The FIFO memory is implemented using two pointers, as shown in Figure 2: a write pointer (tail) and a read pointer (head). The tail pointer tracks the next available location to write data into, while the head

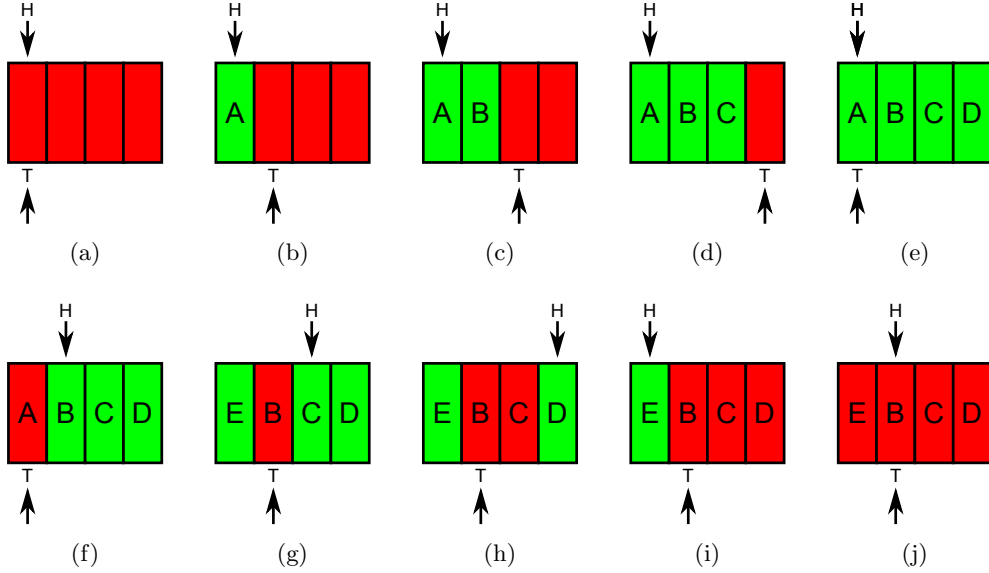


Figure 2: FIFO operation, where red represents invalid data and green represents valid data. (a) The FIFO is initially empty. (b–e) Data A, B, C, and D are sequentially written into the FIFO, moving the tail pointer to the next memory location after each write operation. In (e), all entries are valid and the FIFO is full, with the tail pointer wrapping around to the first memory location. (f) The first read operation outputs data A, moving the head pointer to the next memory location and marking the vacated slot as invalid. (g) Data E is written and data B is read simultaneously. (h–j) Data C, D, and E are read sequentially, leaving the FIFO empty.

pointer tracks the next location to read data from. In the initial empty state, both pointers refer to the same memory location. When a push transaction is accepted (i.e., both `push_valid_i` and `push_grant_o` are asserted high), the incoming data at `push_data_i` is written to the location indicated by the tail pointer, which is then moved to the next memory location. Similarly, when a pop transaction is accepted (i.e., both `pop_valid_o` and `pop_grant_i` are asserted high), the data at the head pointer is forwarded to `pop_data_o` and the head pointer is moved to the next memory location. Both pointers wrap around to the beginning of the memory array once they reach the last slot, implementing a circular buffer structure.

To determine the state of the FIFO, a counter is also maintained internally. The counter is initialized to zero upon reset, reflecting the empty state of the FIFO. Each accepted push transaction increments the counter by one, while each accepted pop transaction decrements it by one. When both push and pop transactions are accepted in the same clock cycle, the counter remains unchanged. The counter is used to control the following outputs: `push_grant_o` is deasserted when the counter is equal to the configured FIFO depth, indicating that the FIFO is full and cannot accept new data; `pop_valid_o` is deasserted when the counter is zero, indicating that the FIFO is empty and there is no data available to be read.

2.2 Parity Checker

Table 2 lists the external interface of the parity checker. It is a purely combinational block that checks the incoming data from the FIFO for parity errors before forwarding it to the output channel. The module supports three configurable parameters: the data width (`DataWidth`); the parity type (`ParityType`), whether even or odd; and the parity bit position (`ParityBit`), whether MSB or LSB. Note that the input data width (`FifoWidth`) is equal to `DataWidth` + 1, where the extra bit is the parity bit.

Table 2: Ports of the parity checker.

Port name	Direction	Width	Description
<code>pop_data_i</code>	in	<code>FifoWidth</code>	data received from the FIFO output
<code>pop_valid_i</code>	in	1	indicates that the FIFO has data available to be read
<code>pop_grant_o</code>	out	1	indicates that the consumer is ready or a parity error is detected
<code>data_o</code>	out	<code>DataWidth</code>	data forwarded to the consumer without parity bit
<code>valid_o</code>	out	1	indicates that the output data is valid and parity is correct
<code>grant_i</code>	in	1	indicates that the consumer is ready to receive data

The parity checker determines whether the incoming data is corrupt by performing a unary XOR reduction on the input data, `pop_data_i`. For even parity, no error is detected when the result is zero; for odd parity, no error is detected when the result is one. The original data is then recovered by stripping the parity bit from either the MSB or LSB position, depending on the configured `ParityBit` parameter, and is forwarded to `data_o`.

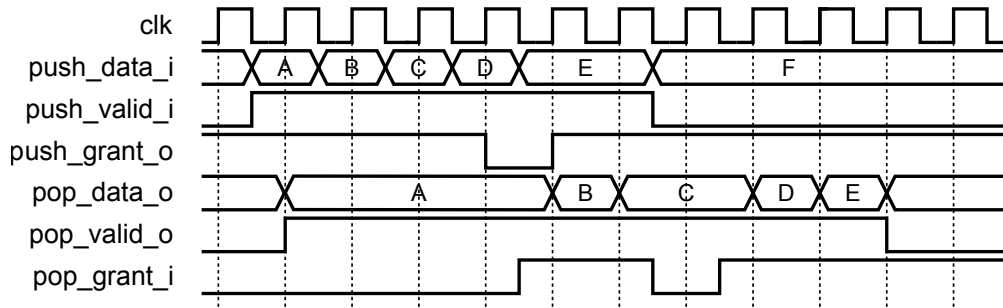
The module also has two output signals that control data flow: `pop_grant_o` and `valid_o`. The `pop_grant_o` signal is asserted in two cases: when the consumer (receiver) is ready to receive data (i.e., `grant_i` is asserted high), or when a parity error is detected, regardless of whether the consumer is ready. In the latter case, the corrupted data is immediately dropped, preventing it from being sampled by the consumer. The `valid_o` signal is only asserted when the incoming data is valid (i.e., `pop_valid_i` is asserted high) and no parity error is detected. This ensures that only valid, error-free data is forwarded to the output channel.

3 Design Verification

The system is described in Verilog. It is configured with the following default parameters: `DataWidth = 32`, `FifoDepth = 128`, `ParityBit = MSB`, `ParityType = Even`. The design is synthesized and implemented in the SkyWater 130nm open-source PDK using the LibreLane RTL-to-GDSII flow for a clock frequency of $50MHz$. It is then simulated using Icarus Verilog to perform post-layout simulation, and the waveforms are viewed using GTKWave. The complete source files, LibreLane scripts, and simulation scripts can be found at the following [link](#).

3.1 FIFO Operation

Figure 3 shows the timing waveform of the FIFO operation, illustrating the example described in Figure 2. In this example, the FIFO has a depth of 4 and a data width of 32 bits. Four data words (A, B, C, and D) are consecutively pushed into the FIFO, filling it to capacity and causing `push_grant_o` to be deasserted. Although data word E is valid on the push interface, it cannot be written since the FIFO is already full. Once the consumer begins reading, `push_grant_o` is reasserted and data word E is written into the FIFO. The data words are then retrieved in the order they were written (A, B, C, D, and finally E), demonstrating the first-in, first-out behavior of the memory. Once all data words have been retrieved, the FIFO becomes empty and `pop_valid_o` is deasserted.



(a) Redrawn for clarity.



(b) Actual waveform from GTKWave.

Figure 3: Timing diagram for FIFO operation.

3.2 Top-Level Operation

The top-level design, including both the FIFO memory and parity checker, is verified using four test scenarios: filling and emptying the FIFO, random traffic at maximum bandwidth, random traffic with random grant, and fault injection. A testbench is implemented to drive the same stimuli to both the design

under test (DUT) and a golden reference model. The reference model generates the expected outputs, which are then used by the scoreboard for checking. The scoreboard compares the outputs of the DUT against the expected outputs and reports any mismatches.

3.2.1 FIFO Fill and Empty

This test case handles when the FIFO is filled to capacity and then emptied afterwards. An example of the result is shown below. All data entries are transmitted, and every entry is correctly received. No data is lost, corrupted, or mishandled during either the fill or empty phase. There are also no incorrect grants or incorrect valids, confirming that the output control signals are asserted and deasserted at the correct times.

```

-----
Number of data transmitted (good and bad) = 128
Number of good data transmitted          = 128
Number of good data correctly received   = 128
Number of good data incorrectly received = 0
Number of bad data transmitted           = 0
Number of bad data correctly dropped     = 0
Number of bad data incorrectly dropped   = 0
Number of incorrect grants               = 0
Number of incorrect valids              = 0
-----
PASS

```

3.2.2 Random Traffic at Maximum Bandwidth

This test case handles random traffic at maximum bandwidth, where `pop_grant_i` is always asserted. In this case, the consumer accepts every data word as soon as it is available. A sample of the result is shown below. Similar to the previous test case, all data are accurately received with no errors.

```

-----
Number of data transmitted (good and bad) = 77805
Number of good data transmitted          = 77805
Number of good data correctly received   = 77805
Number of good data incorrectly received = 0
Number of bad data transmitted           = 0
Number of bad data correctly dropped     = 0
Number of bad data incorrectly dropped   = 0
Number of incorrect grants               = 0
Number of incorrect valids              = 0
-----
PASS

```

3.2.3 Random Traffic with Random Grant

This test case handles random traffic where `pop_grant_i` toggles randomly between high and low, unlike the previous case where it is always asserted. This tests whether the FIFO correctly holds data when the consumer is not ready and properly forwards it when the consumer becomes ready. For this test case, the FIFO is fully emptied before the results are recorded. A sample of the result is shown below, with all data accurately received and no errors reported.

```

-----
Number of data transmitted (good and bad) = 18172
Number of good data transmitted          = 18172
Number of good data correctly received   = 18172
Number of good data incorrectly received = 0
Number of bad data transmitted           = 0
Number of bad data correctly dropped     = 0
Number of bad data incorrectly dropped   = 0
Number of incorrect grants               = 0
Number of incorrect valids              = 0
-----
PASS

```

3.2.4 Fault Injection

To test the parity checker, random bit errors are injected into the data stream, with at most one fault per transaction. The checker monitors the output of the parity checker and verifies two conditions: first, that all corrupted data is detected and dropped, and never forwarded to the consumer; and second, that all valid data words are correctly forwarded to the consumer in order. This test is repeated across all four parity configurations to ensure correctness regardless of the parity setting. Note that the traffic patterns described in the previous subsections assume error-free data, while this test specifically validates the ability of the parity checker to detect and drop corrupted data.

An example of the result is shown below with a specified fault rate of 40%. Of the total transactions, approximately 40% are corrupted and the remaining 60% are valid. All valid data is correctly received, and all corrupted data is correctly detected and dropped.

```

-----
Number of data transmitted (good and bad) = 44042
Number of good data transmitted           = 26454
Number of good data correctly received    = 26454
Number of good data incorrectly received  = 0
Number of bad data transmitted            = 17588
Number of bad data correctly dropped       = 17588
Number of bad data incorrectly dropped     = 0
Number of incorrect grants                = 0
Number of incorrect valids                = 0
-----

```

PASS

3.3 Hardware Implementation

The design is also implemented on a Digilent Arty A7-35T FPGA board, with the testing setup shown in Figure 4. A MicroBlaze soft-core processor is integrated to generate input stimuli and verify the DUT outputs. Input stimuli are stored in a dual-port block RAM (BRAM), with one port connected to the processor and the other connected to a custom memory controller responsible for streaming the stimuli to the DUT. The stimuli are packed into 32-bit words, with one bit each allocated to `valid_i` and `grant_i` and the remaining 30 bits allocated to `data_i`. Consequently, `FifoWidth` is set to 30 bits for this setup. The `FifoDepth` is also set to 64 to fit within the available resources of the target board.

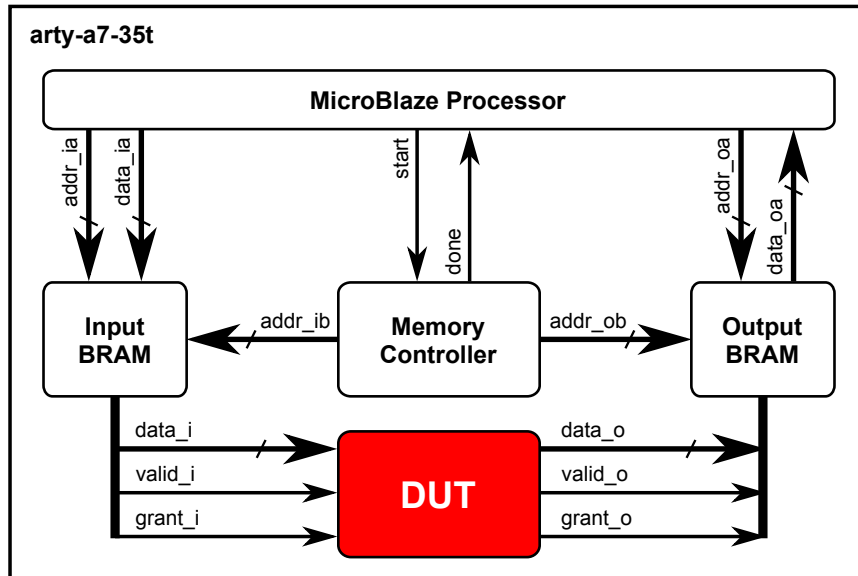


Figure 4: FPGA testing setup showing the DUT and associated test peripherals, including the MicroBlaze processor, dual-port input and output BRAMs, and memory controller. Legend on the BRAM ports: i/o denotes input/output BRAM; a/b denotes port A/B.

The processor first writes all input stimuli to the BRAM. Once the write operation is complete, it asserts a `start` signal, causing the memory controller to begin reading from the BRAM and driving the `data_i`,

`valid_i`, and `grant_i` signals to the DUT every clock cycle. On the output side, a second dual-port BRAM captures the corresponding `data_o`, `valid_o`, and `grant_o` output signals every clock cycle during processing.

After all stimuli have been applied, the memory controller asserts a `done` signal, which triggers an interrupt to the processor. Upon receiving the interrupt, the processor reads the contents of the output BRAM and compares them against the golden output to identify any mismatches. The comparison result is then transmitted via UART and displayed on a terminal.

The design is evaluated using the same four test scenarios described in the previous subsection. As shown in Figure 5, it successfully passed all test scenarios, demonstrating correct functionality of the design in hardware.

```

COM4 - PuTTY
-----
FIFO fill and empty
-----
Number of data transmitted (good and bad) = 64
Number of good data transmitted = 64
Number of good data correctly received = 64
Number of good data incorrectly received = 0
Number of bad data transmitted = 0
Number of bad data correctly dropped = 0
Number of bad data incorrectly dropped = 0
Number of incorrect grants = 0
Number of incorrect valids = 0
-----
PASS
-----

Random traffic, maximum bandwidth
-----
Number of data transmitted (good and bad) = 988
Number of good data transmitted = 988
Number of good data correctly received = 988
Number of good data incorrectly received = 0
Number of bad data transmitted = 0
Number of bad data correctly dropped = 0
Number of bad data incorrectly dropped = 0
Number of incorrect grants = 0
Number of incorrect valids = 0
-----
PASS
-----

COM4 - PuTTY
-----
Random traffic, random grant
-----
Number of data transmitted (good and bad) = 1046
Number of good data transmitted = 1046
Number of good data correctly received = 1046
Number of good data incorrectly received = 0
Number of bad data transmitted = 0
Number of bad data correctly dropped = 0
Number of bad data incorrectly dropped = 0
Number of incorrect grants = 0
Number of incorrect valids = 0
-----
PASS
-----

Random traffic, fault injection
-----
Number of data transmitted (good and bad) = 1026
Number of good data transmitted = 587
Number of good data correctly received = 587
Number of good data incorrectly received = 0
Number of bad data transmitted = 439
Number of bad data correctly dropped = 439
Number of bad data incorrectly dropped = 0
Number of incorrect grants = 0
Number of incorrect valids = 0
-----
PASS
-----

```

Figure 5: Terminal output.

4 Conclusion

This paper presents the design, implementation, and verification of a synchronous FIFO memory with parity checking. The design is verified using a set of directed and random test scenarios, demonstrating correct FIFO operation, parity error detection, and configurability across different parity settings. To further validate the design, it is prototyped on a Xilinx FPGA platform, where a soft-core processor is integrated to drive input stimuli and compare the design outputs against a golden reference. The results confirm the correct functionality of the design in both simulation and hardware implementation.